Name: Arnav Dhole
Roll No: 12
Class: D12C

J Flap

Theory

1. Introduction to Finite Automata

Finite automata are mathematical models used to recognize patterns in strings of symbols. They are widely used in compiler design, lexical analysis, pattern matching, and formal language theory.

A finite automaton consists of a finite number of states, transitions between states, an initial state, and one or more final states. It processes an input string symbol by symbol and determines whether the string belongs to a particular language.

The alphabet is represented by Σ, which is the finite set of input symbols. In these practicals, the alphabet is generally $\Sigma = \{0, 1\}$.

2. Deterministic Finite Automaton (DFA)

A Deterministic Finite Automaton (DFA) is a finite automaton in which every state has exactly one defined transition for each input symbol in the alphabet. Therefore, for a given input string, there is only one possible path through the automaton.

A DFA is formally represented as:

$$M = (Q, \Sigma, \delta, q_0, F)$$

Where:

- Q is the finite set of states.
- Σ is the input alphabet.
- δ is the transition function.
- q_0 is the initial state.
- F is the set of final or accepting states.

A string is accepted if, after processing the entire input, the automaton reaches a final state. Otherwise, the string is rejected.

Applications in the practicals: DFA is used to recognize strings ending with `0`, strings containing `01` as a substring, strings containing an even number of `0`s, and strings ending with `01`.

3. Nondeterministic Finite Automaton (NFA)

A Nondeterministic Finite Automaton (NFA) is a finite automaton in which a state may have multiple transitions for the same input symbol, or no transition for a particular symbol. This allows multiple possible paths to be explored for the same input string.

An NFA is formally represented as:

$$M = (Q, \Sigma, \delta, q_0, F)$$

The main difference between a DFA and an NFA lies in their transition functions. In an NFA, a state and input symbol can lead to a set of possible next states.

An input string is accepted if at least one possible path processes the entire string and ends in a final state. If no path reaches a final state after processing the complete string, the string is rejected.

Applications in the practicals: NFA is used to recognize strings ending in either `00` or `11`, strings containing `101` as a substring, strings containing either `01` or `10` as a substring, and strings ending in either `01` or `10`.

4. Difference Between DFA and NFA

DFA	NFA
Has exactly one transition for each state and input symbol.	May have multiple transitions for a state and input symbol.
Has only one possible computation path for an input.	Can have multiple possible computation paths.
Accepts if its single path ends in a final state.	Accepts if at least one path ends in a final state.
Transition function returns one state.	Transition function returns a set of states.
Execution follows a unique path.	Acceptance depends on the existence of a successful path.

Both DFA and NFA recognize regular languages and have equivalent expressive power.

5. Substrings and Suffixes

A substring is a sequence of consecutive symbols that appears anywhere within a string. For example, `101` is a substring of `11010`.

A suffix is a sequence of symbols appearing at the end of a string. For example, `01` is a suffix of `11001`.

These concepts are important when constructing automata:

- Containing a substring: The automaton accepts a string if the required pattern appears anywhere in it.
- Ending with a suffix: The automaton accepts a string only if the required pattern occurs at the end of the complete input.

For example, an automaton that recognizes strings containing `01` may accept `0100`, whereas an automaton that recognizes strings ending with `01` must reject `0100`.

6. Conversion of NFA into DFA

An NFA can be converted into an equivalent DFA using the subset construction method. The purpose of this method is to create a deterministic automaton that accepts exactly the same language as the original NFA.

The main steps are:

1. The initial state of the DFA is formed from the initial state of the NFA, including its epsilon-closure if epsilon transitions are present.
2. For each DFA state and each input symbol, determine all NFA states reachable using that symbol.
3. Treat each resulting set of NFA states as a single state in the DFA.
4. Repeat the process until no new states are generated.
5. Mark a DFA state as final if its set contains at least one final state of the original NFA.

The resulting DFA may contain more states than the original NFA. However, both automata recognize the same language.

In the final two practicals of the uploaded document, this conversion is applied to automata recognizing strings containing `01` and `10` as substrings, respectively.

7. JFLAP Software

JFLAP (Java Formal Languages and Automata Package) is an educational software tool used to design, simulate, and analyze finite automata and other computational models.

It provides a graphical interface for creating states, adding transitions, marking initial and final states, testing input strings, and converting NFAs into equivalent DFAs.

The general procedure for these practicals is:

1. Create the required states and transitions.
2. Mark the initial state and accepting states.
3. Enter a test string in the input simulation tool.
4. Execute the simulation and observe whether the string is accepted or rejected.
5. For conversion practicals, use JFLAP's conversion facility to obtain an equivalent DFA and test its behavior.

8. Acceptance and Rejection of Strings

A string is accepted by a finite automaton when the entire input has been processed and the

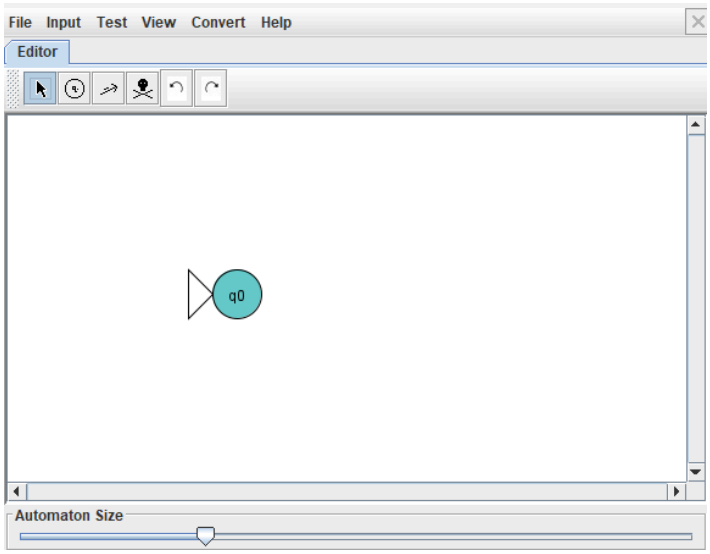
automaton finishes in an accepting state. A string is rejected when the final state reached is not accepting, or when no valid computation path processes the complete input.

Testing both accepted and rejected strings verifies whether the automaton follows the specified language rules.

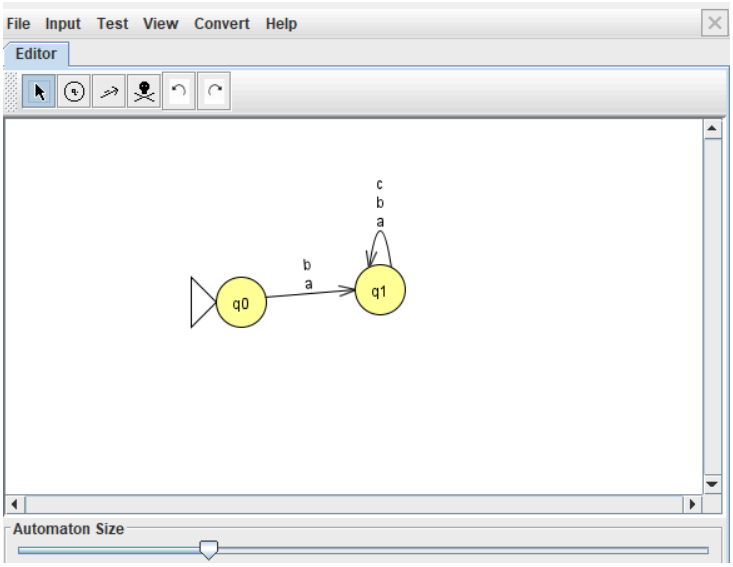
For example, in the practical involving strings ending with `0`, the input `0110010` is accepted because its last symbol is `0`, while `1001101` is rejected because its last symbol is `1`.

Example:

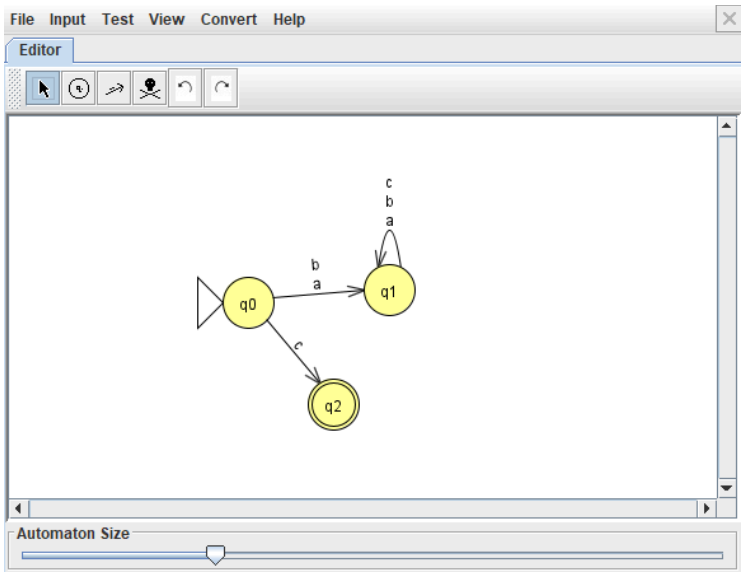
Step 1: Create initial state q0



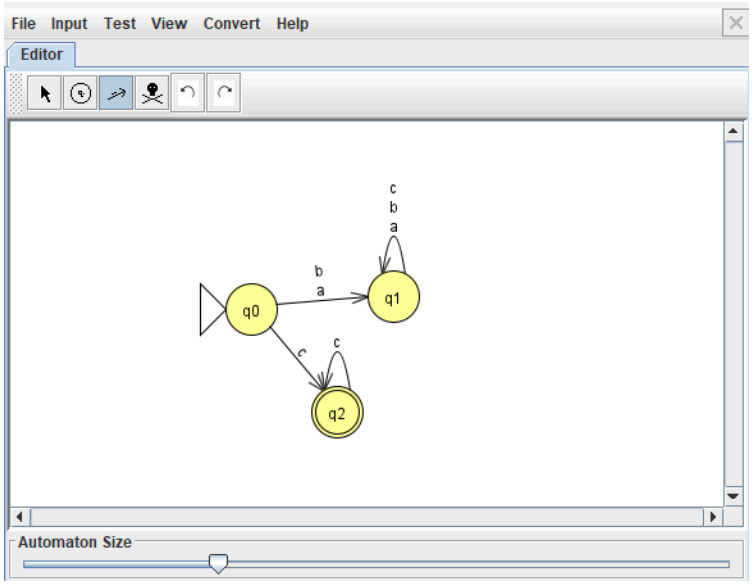
Step 2: Create a new state, q1, as the destination of a or b from q0 and which traps all subsequent substrings



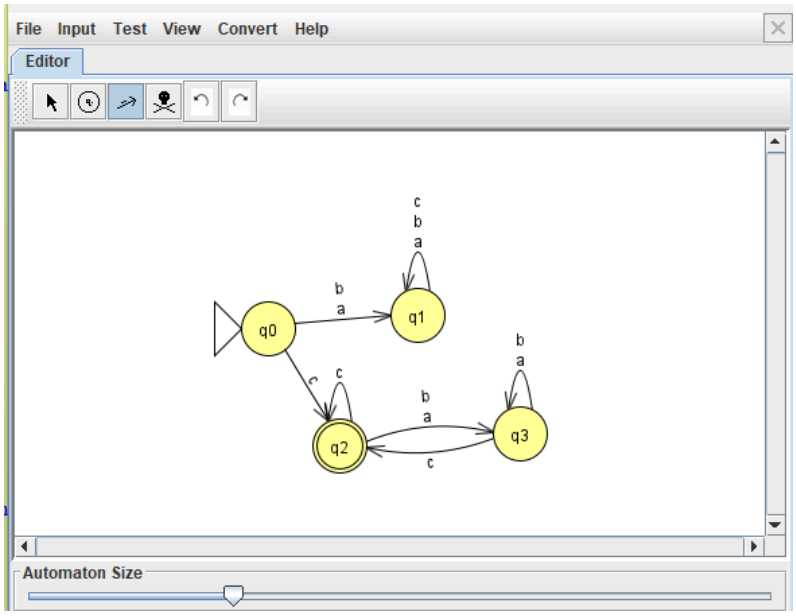
Step 3: Create a new state, q2, as the destination of transition c from q0 and which is an accept state



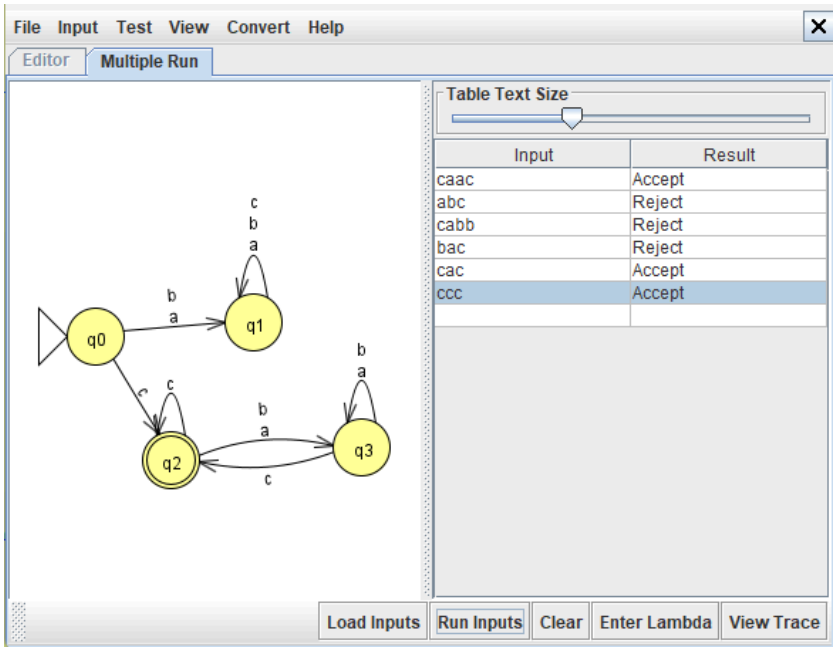
Step 4: Add a c transition to and from q2



Step 5: Add state q3 with a transition of a or b from q2. Add transitions to remain in q3 for a or b. Add transition from q3 back to q2 for c



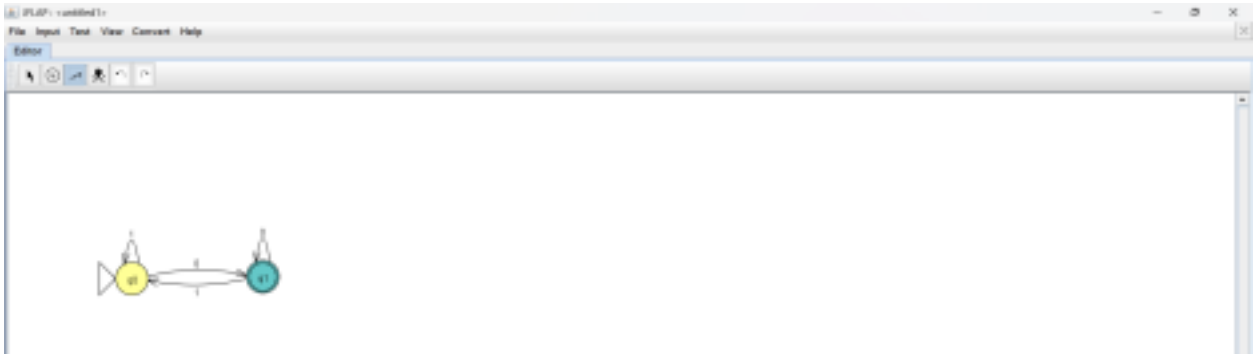
Testing with inputs:



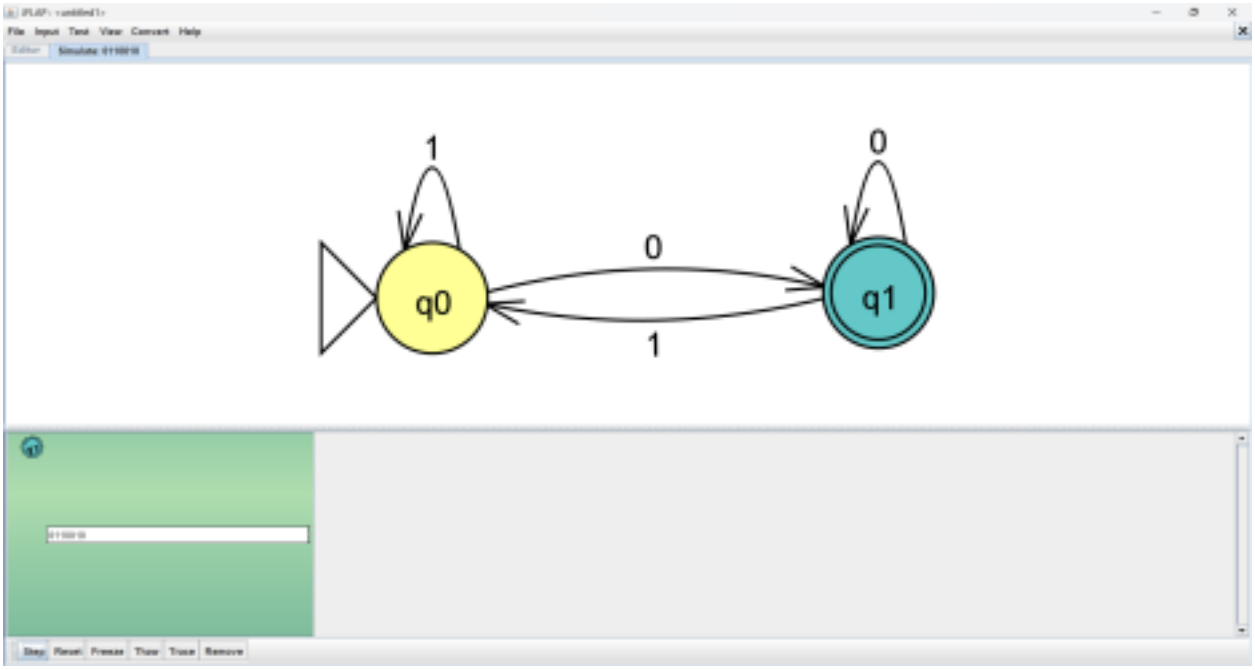
Therefore the DFA accepts all strings that start and end with c

1. **Construct a DFA over Σ = {0,1} that accepts all strings ending with 0.**

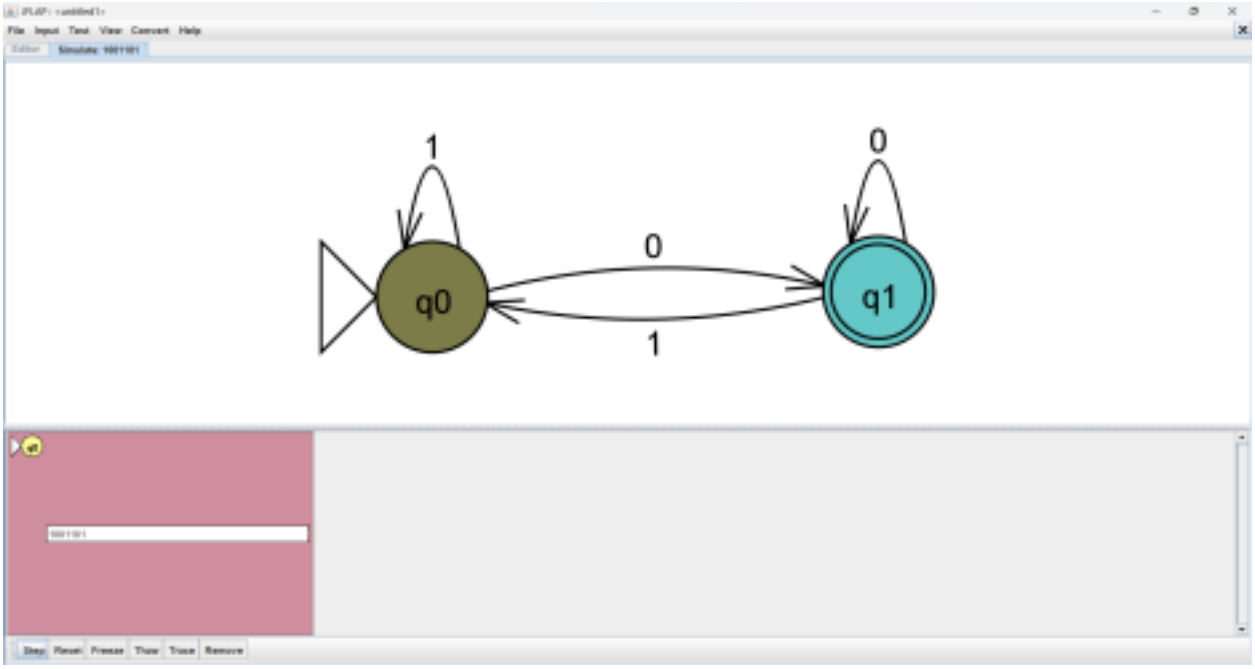
   Automaton:



Test Cases:

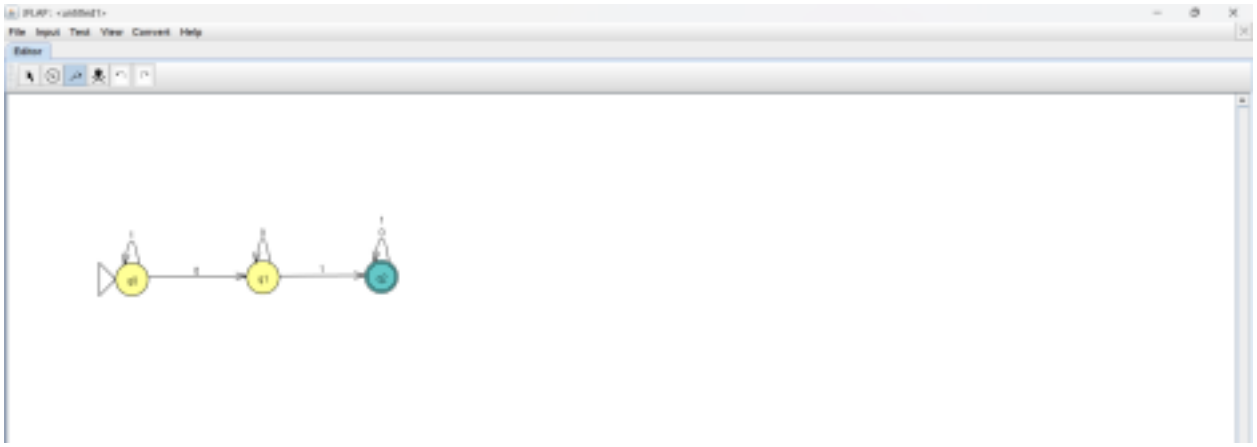


1. *Input: 0110010 - accepted*
2. *Input: 1001101 - rejected*



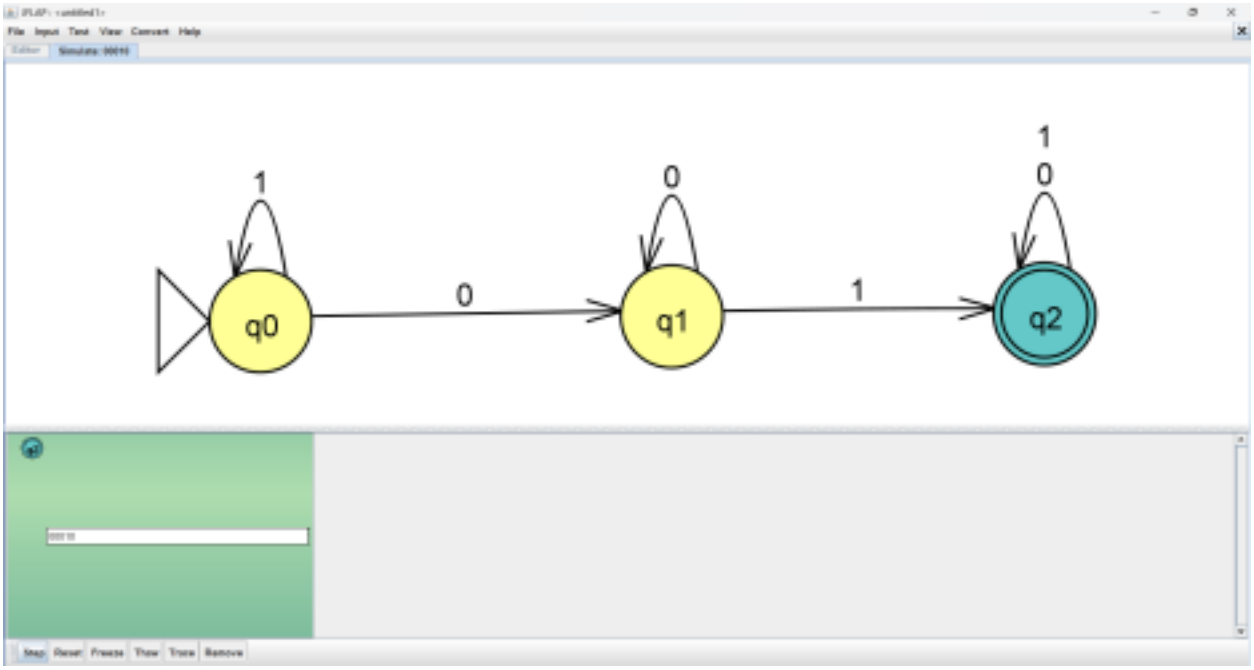
2. Construct a DFA over Σ = {0,1} that accepts all strings containing 01 as a substring.

Automaton:

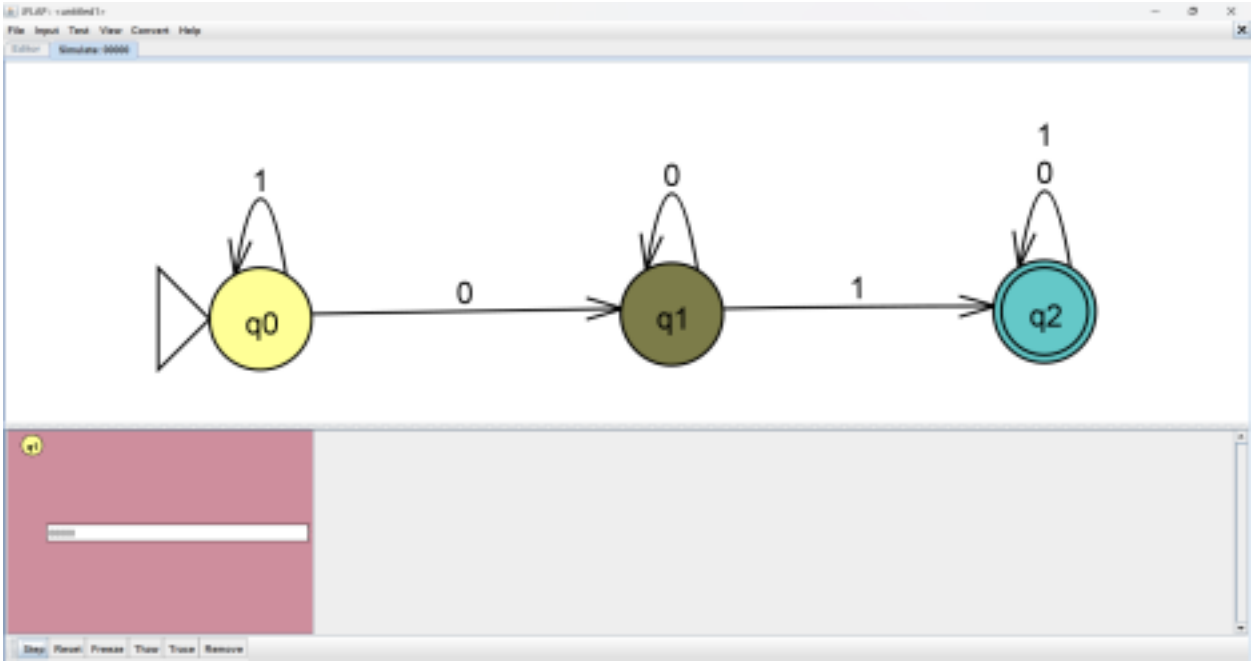


Test Cases:

1. Input: 00010-accepted

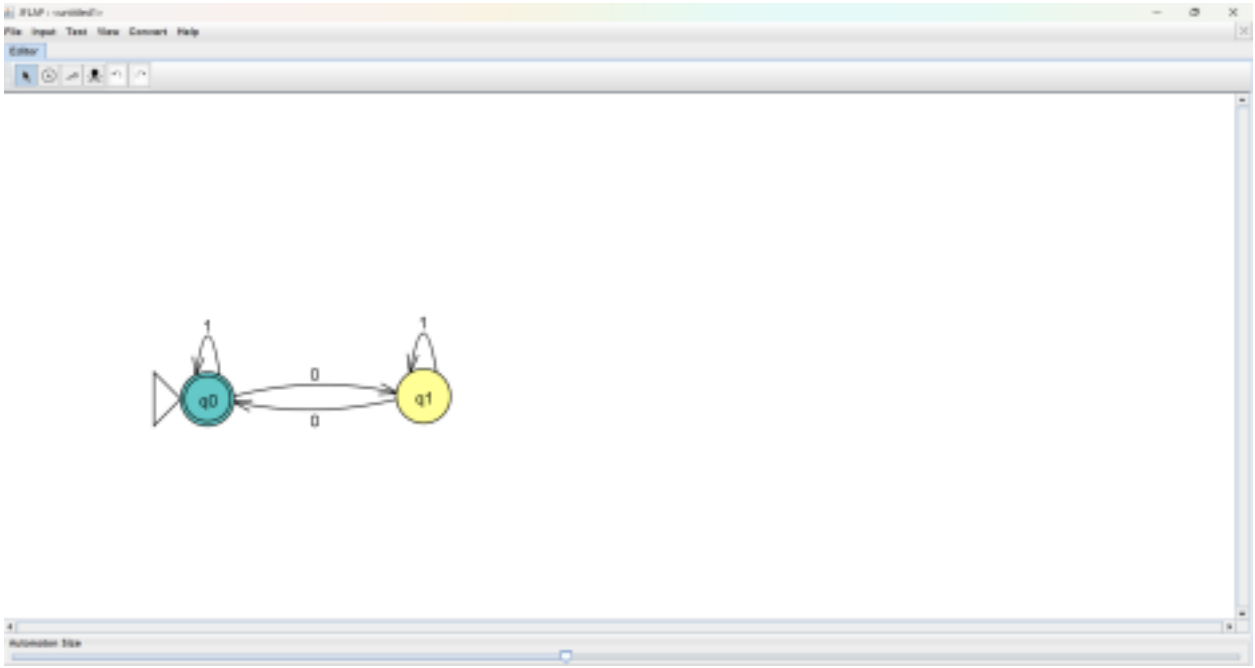


2. Input: 00000-rejected



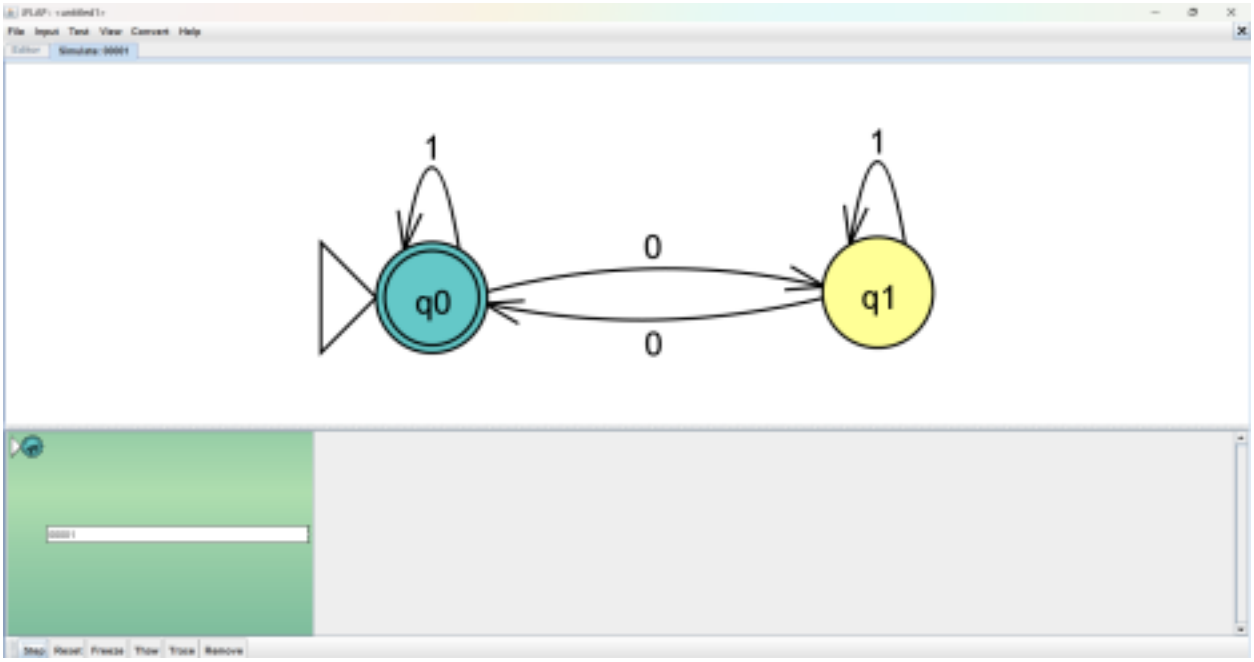
3. Construct a DFA over Σ ={0,1} that accepts strings containing an even number of 0s.

Automaton:

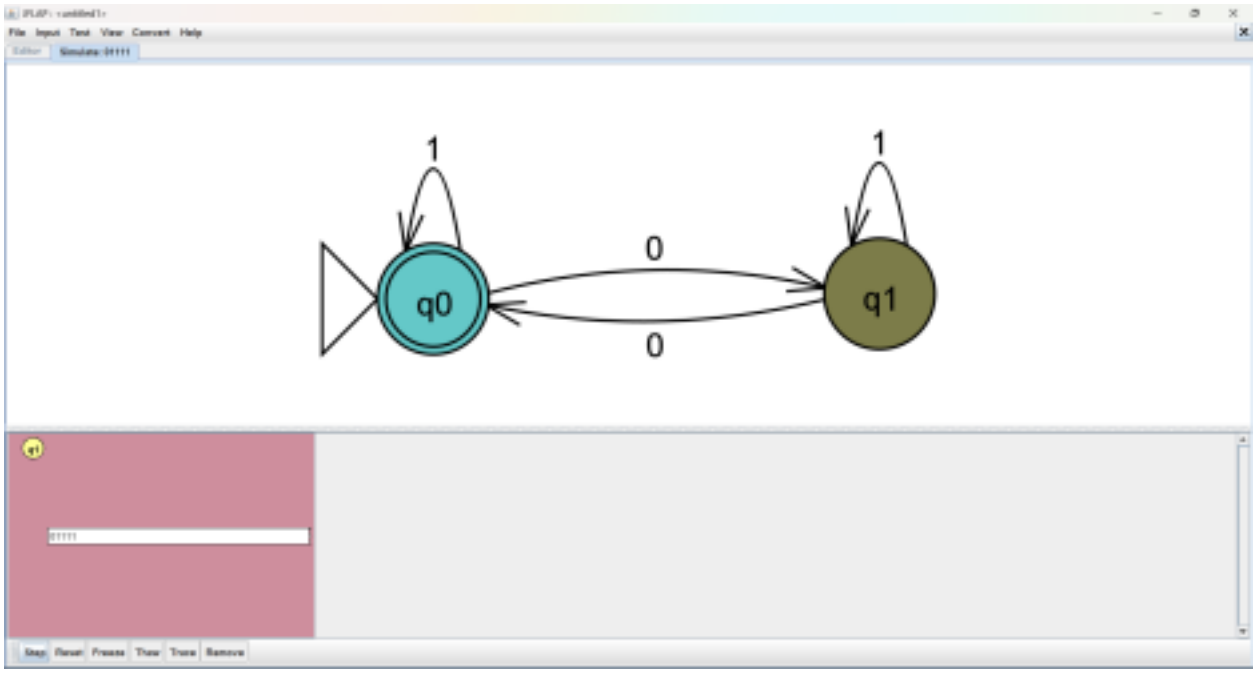


Test Cases:

1. Input: 00001 - accepted

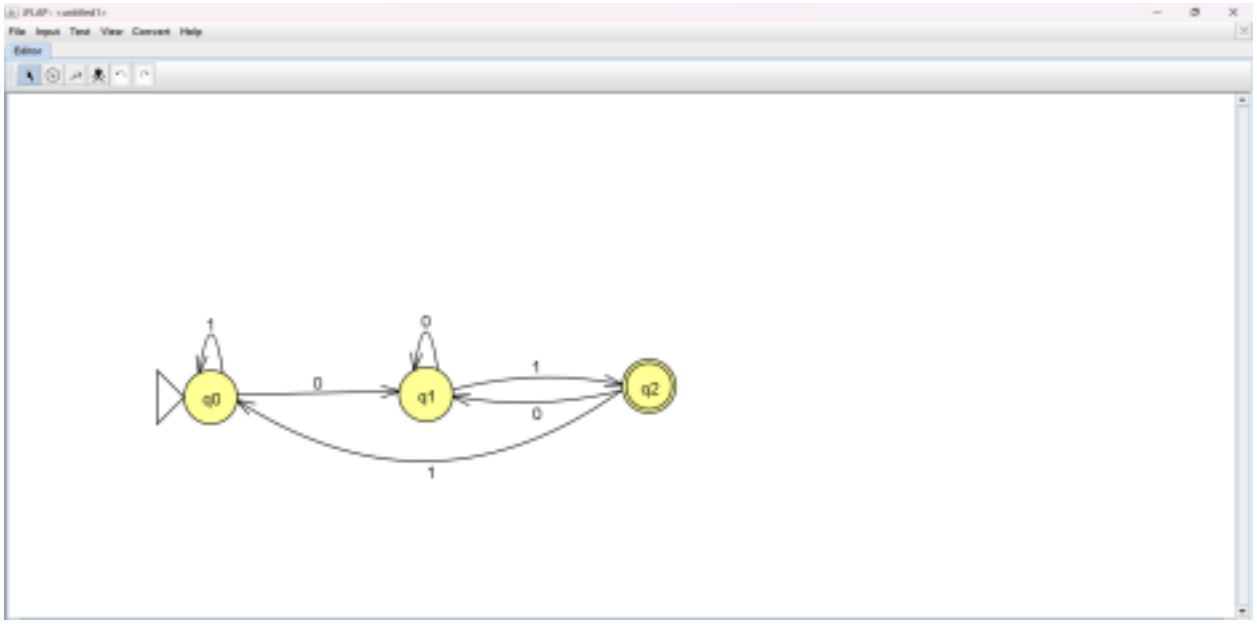


2. Input: 01111 - rejected



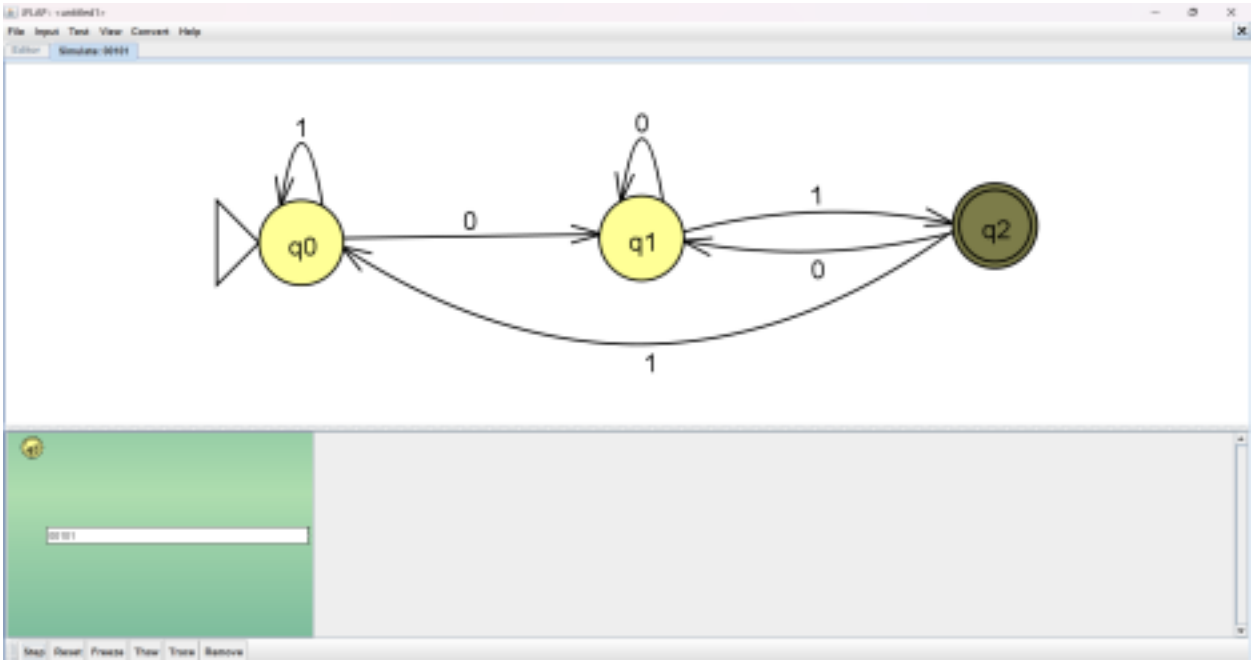
4. Construct a DFA over Σ = {0,1} that accepts all strings ending with 01.

Automaton:

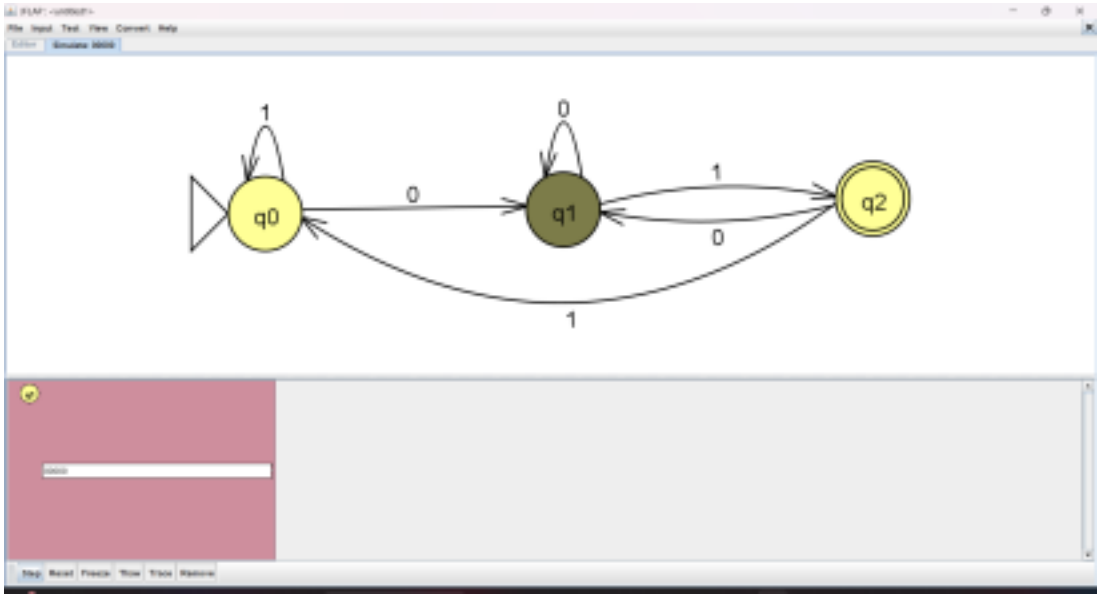


Test Cases:

1. Input: 00101 - accepted

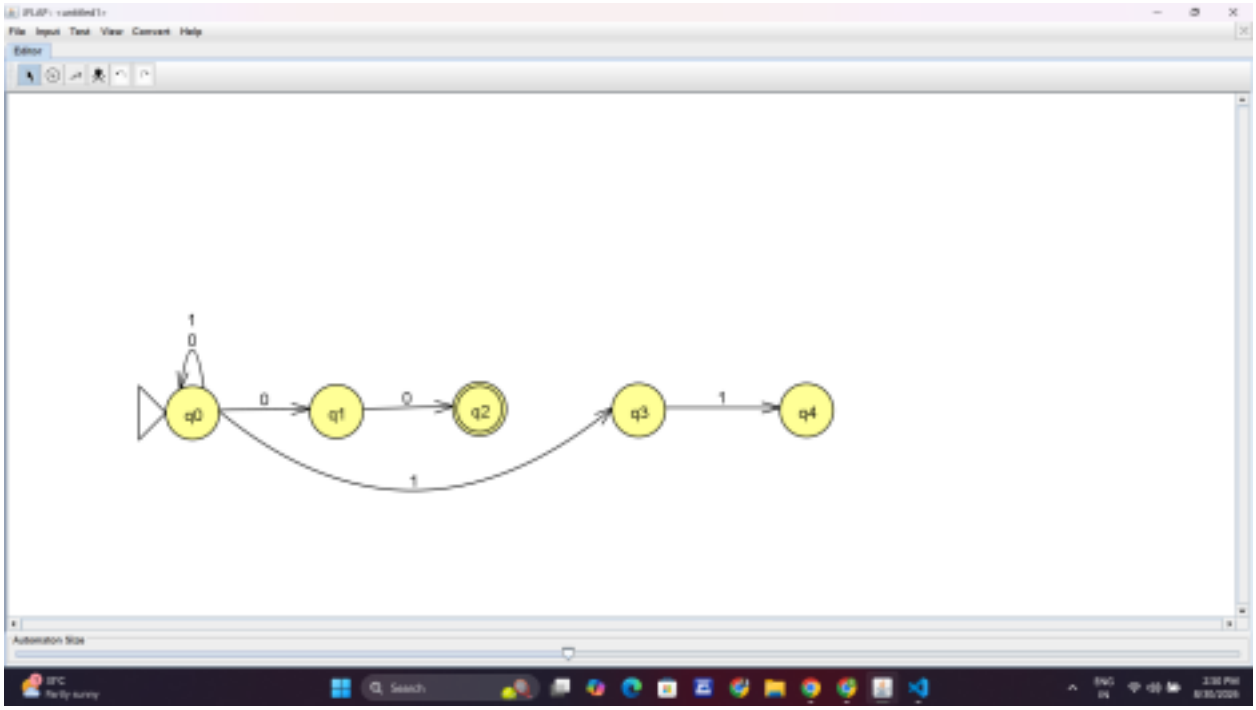


2. Input: 00000 - rejected



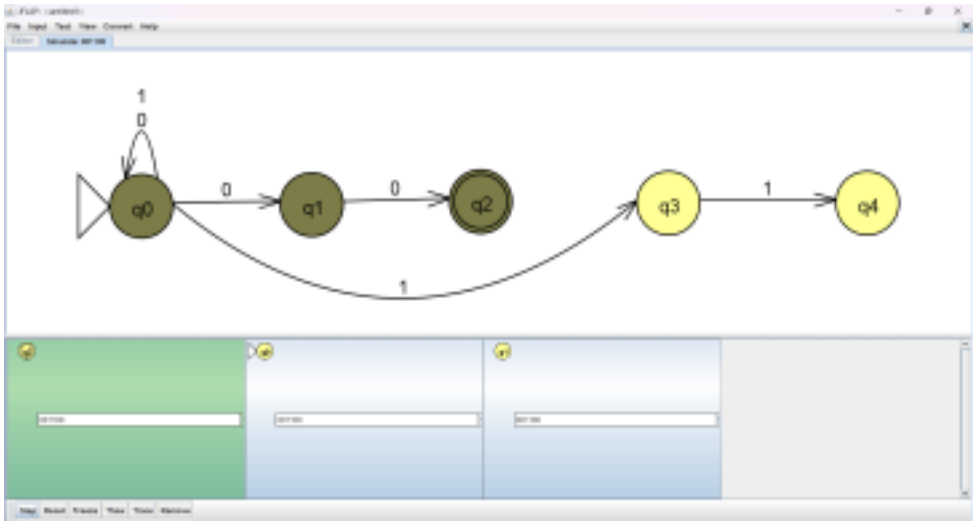
5. Construct an NFA over Σ = {0,1} that accepts all strings ending with either 00 or 11.

Automaton:

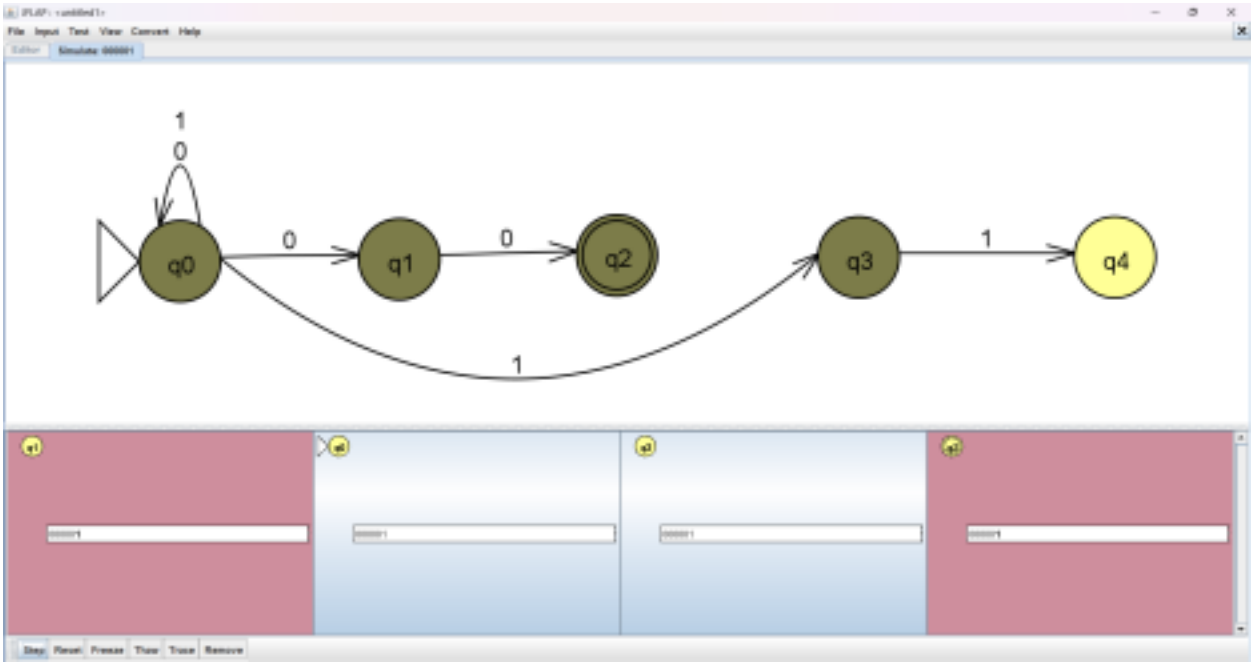


Test Cases:

1. Input: 001010- accepted

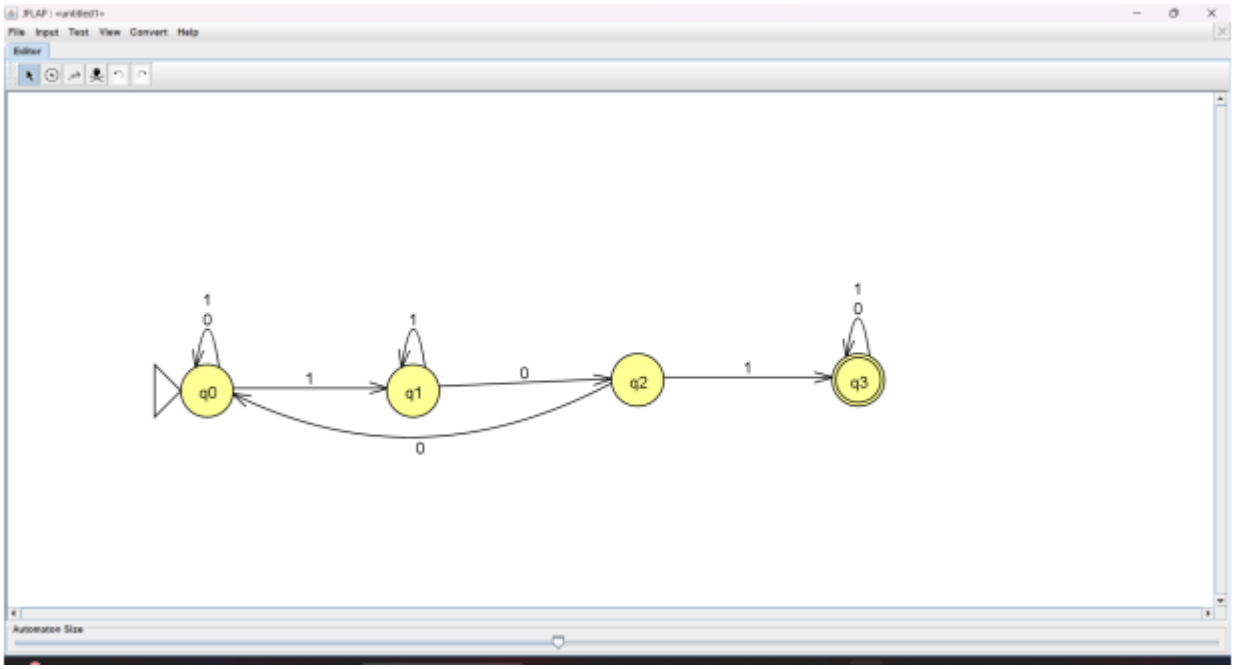


2. Input: 000000 - rejected



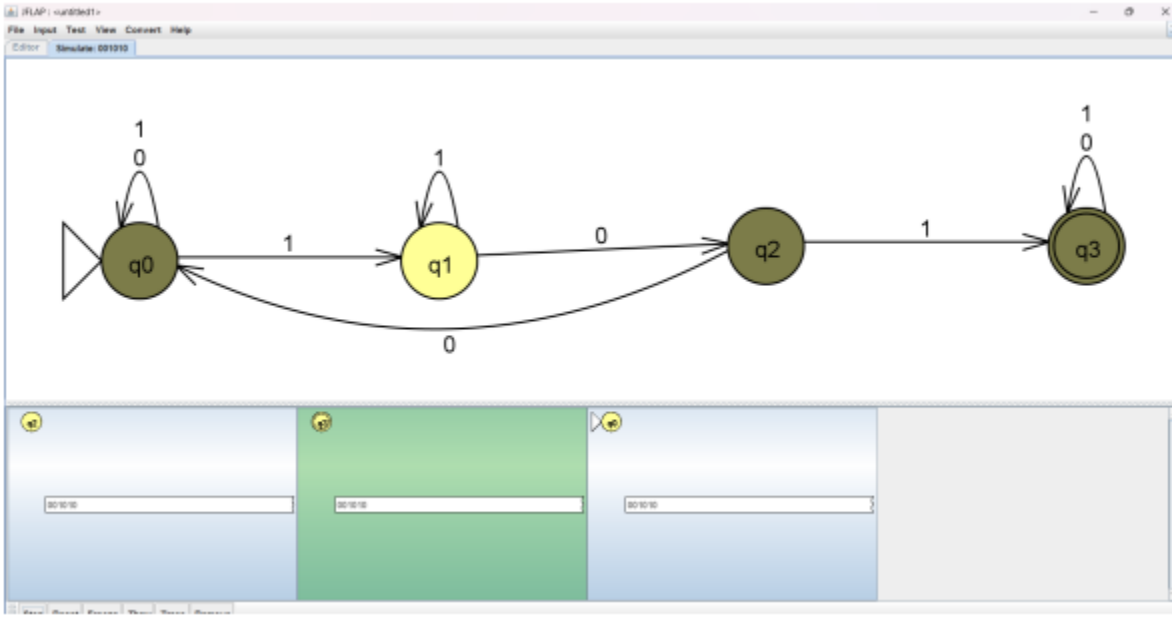
6. Construct an NFA over Σ = {0,1} that accepts all strings containing 101 as a substring

Automaton:

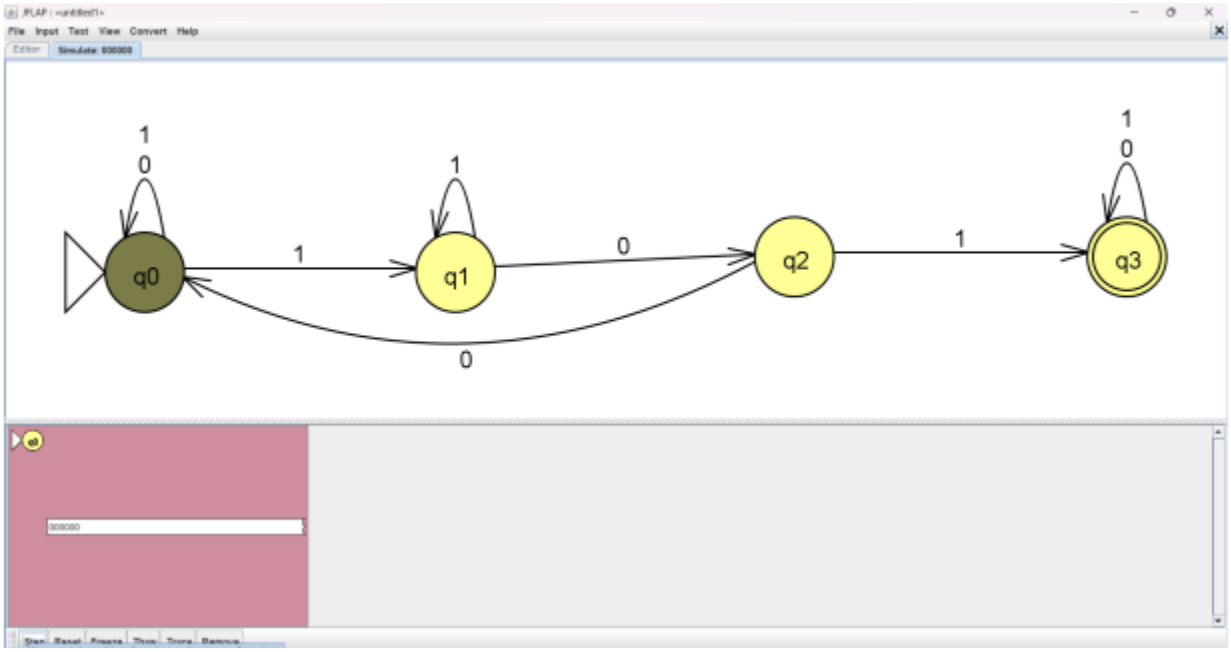


Test Cases:

1. Input: 001100 - accepted

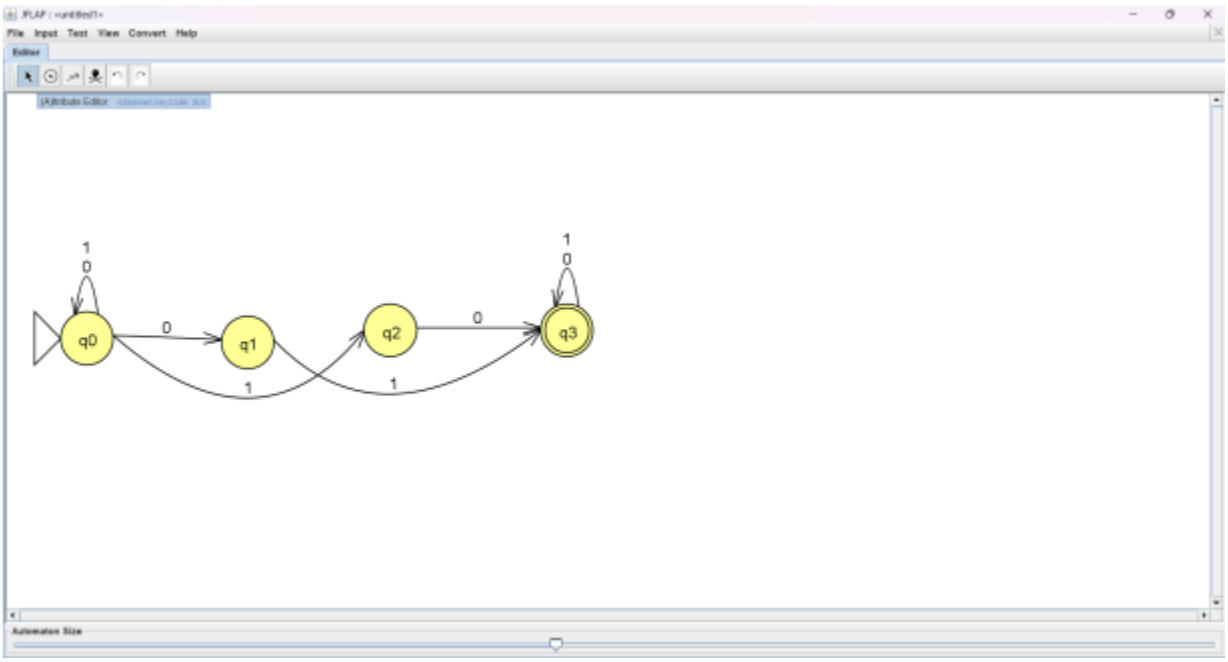


2. Input: 000001 - rejected



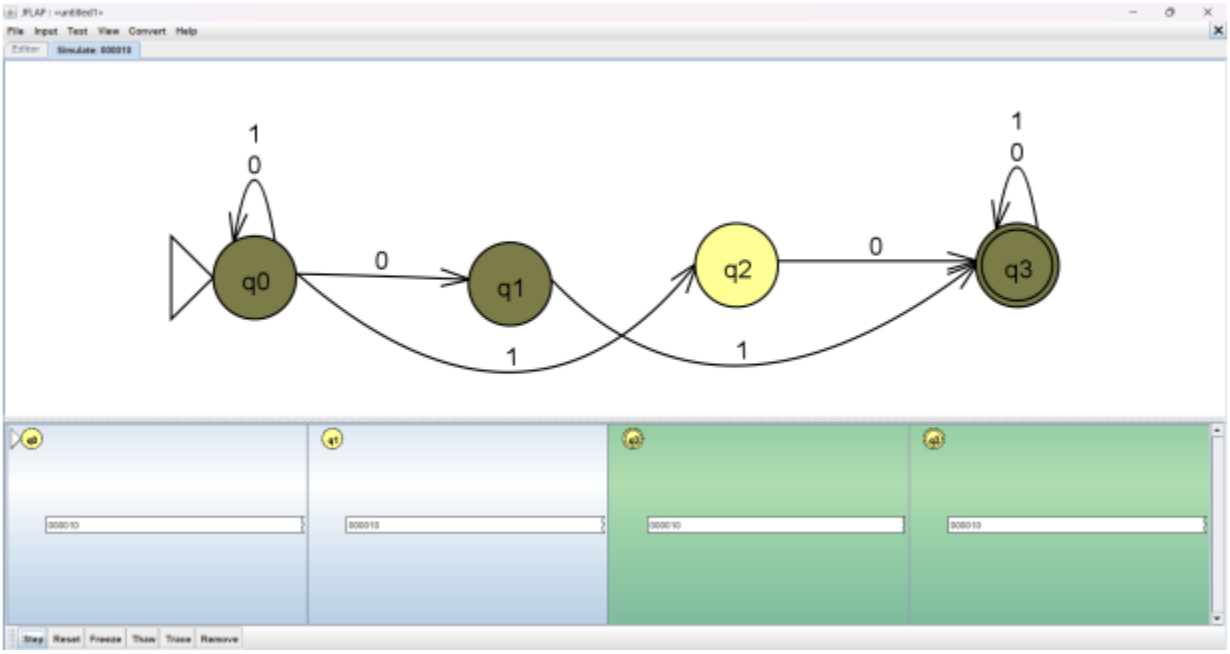
7. Construct an NFA over Σ = {0,1} that accepts all strings containing either 01 or 10 as a substring.

Automaton:

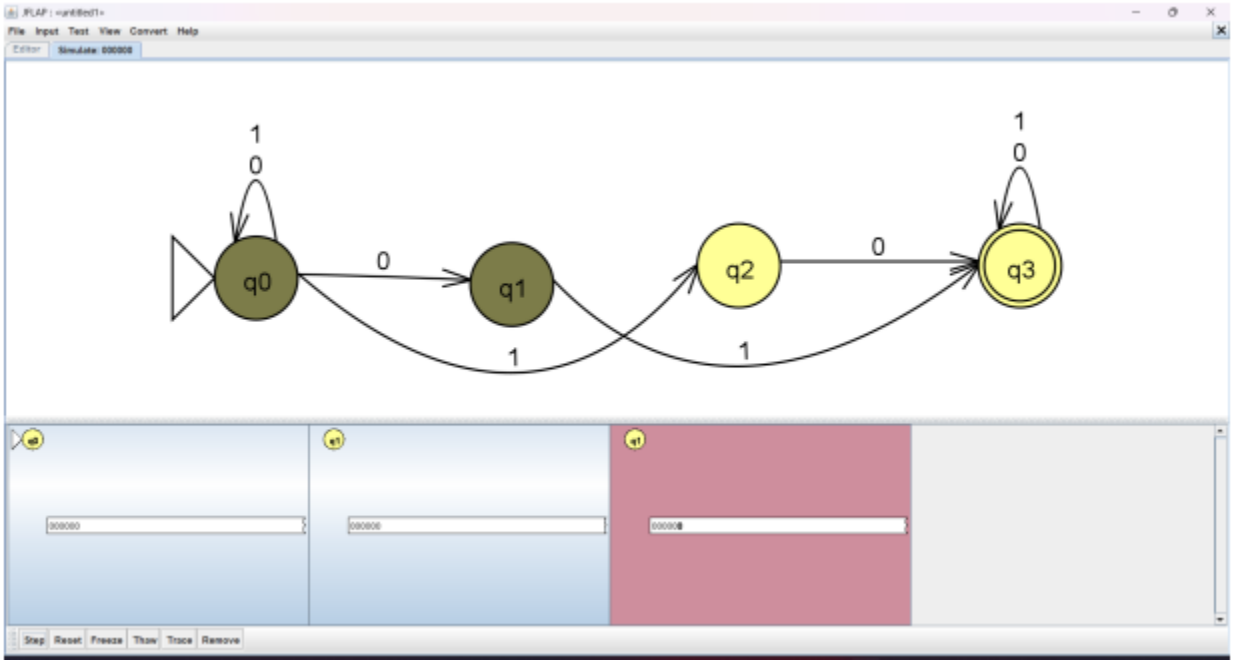


Test Cases:

1. *Input: 000010 - accepted*

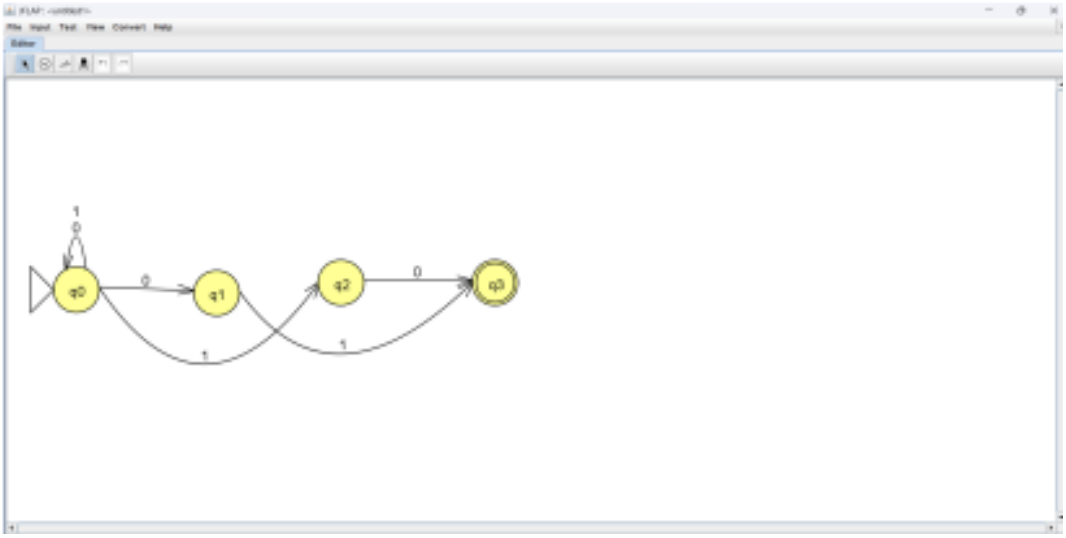


2. *Input: 000000 - rejected*



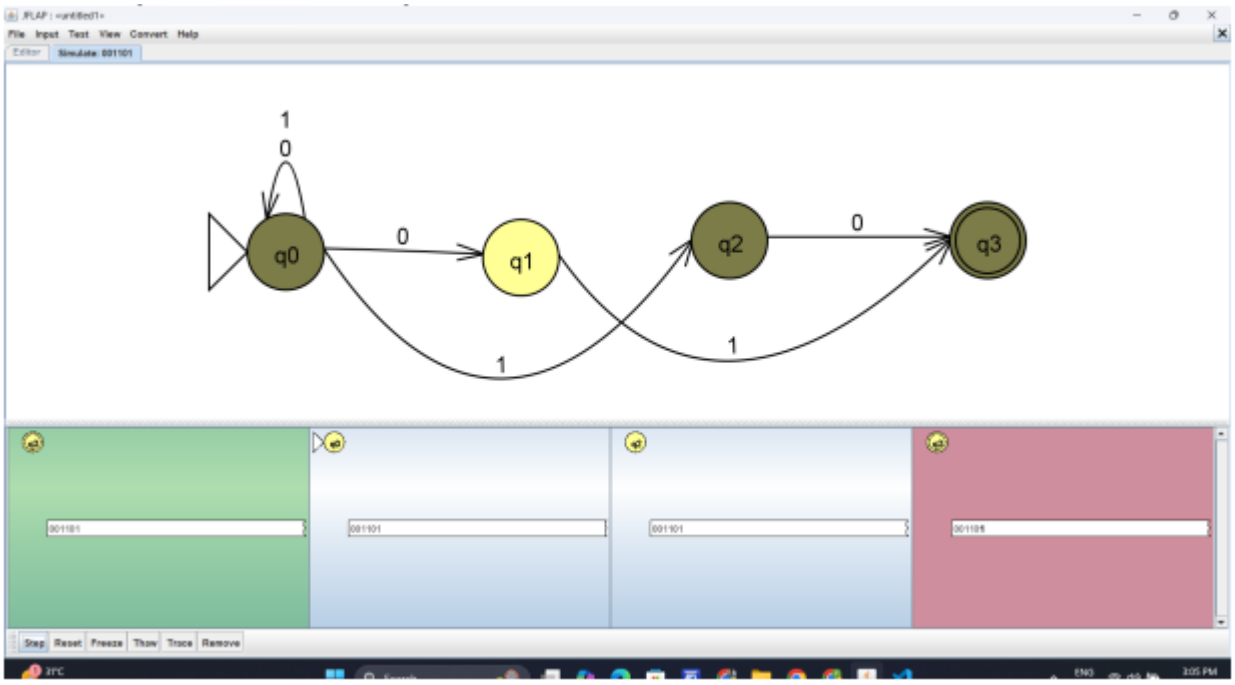
8. Construct an NFA over Σ = {0,1} that accepts all strings ending in either 01 or 10.

Automaton:

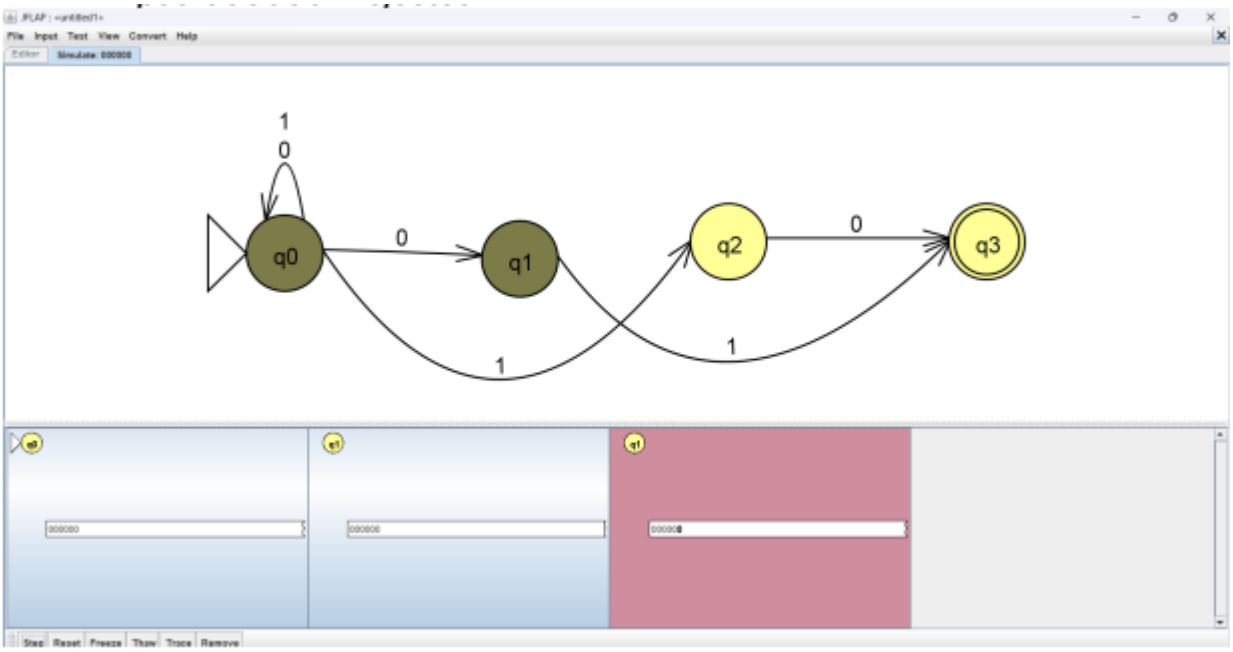


Test Cases:

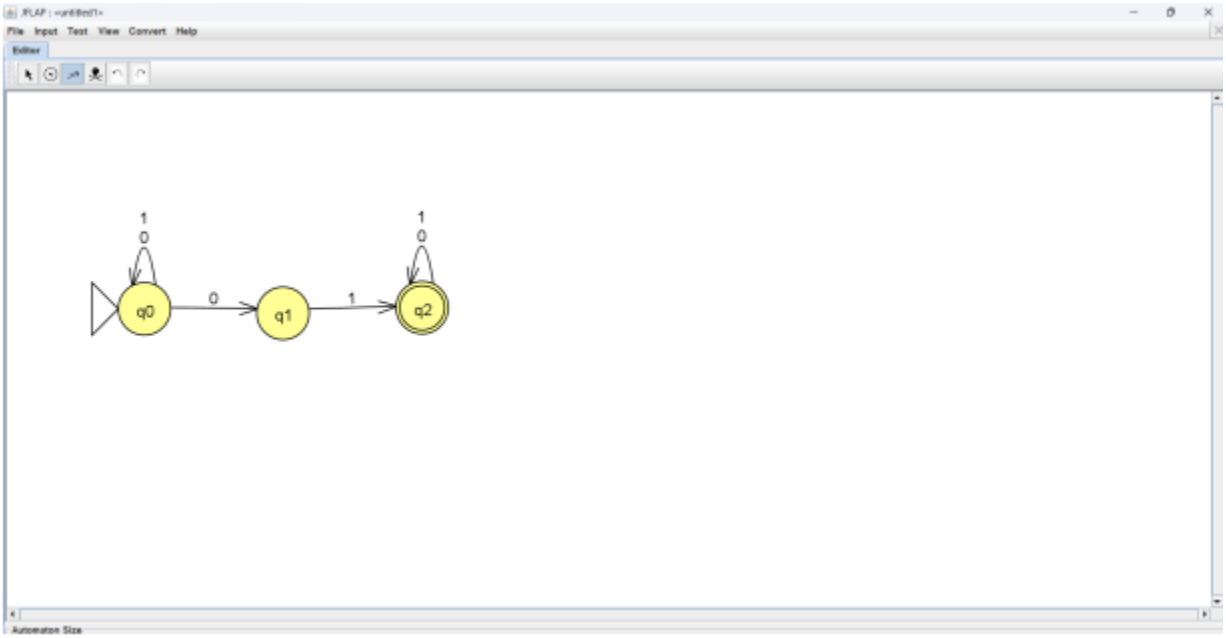
1. *Input: 001101 - accepted*



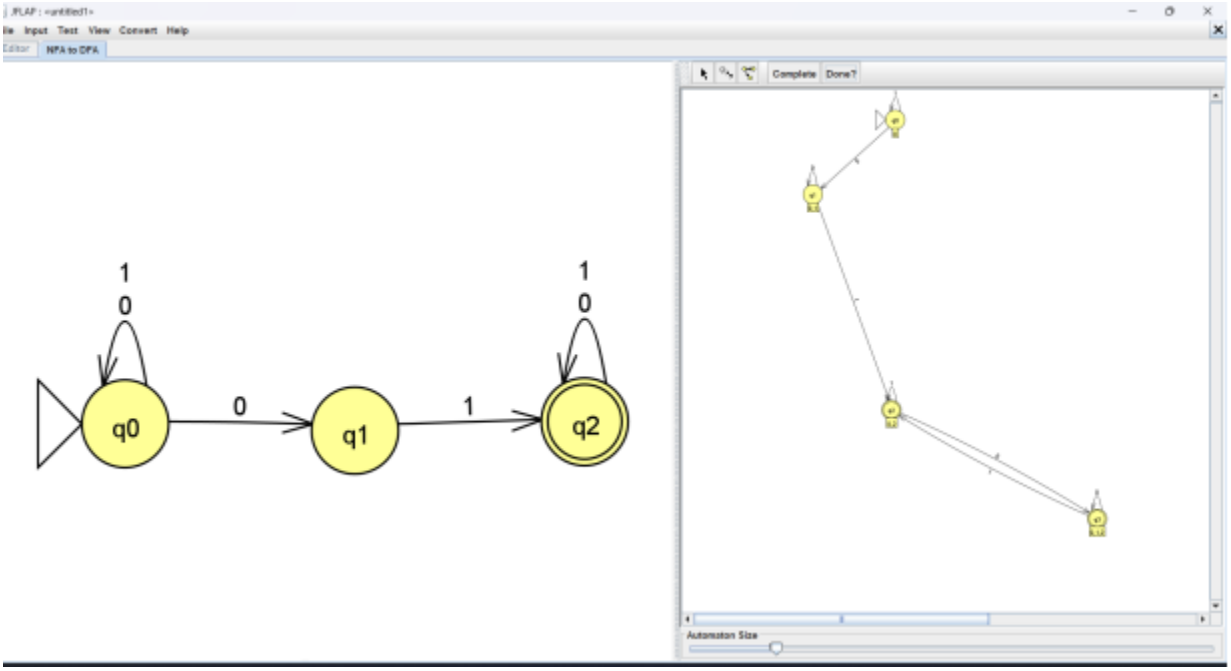
2. Input: 000000 - rejected



9. Construct the given NFA and convert it into an equivalent DFA Substrings containing 01.

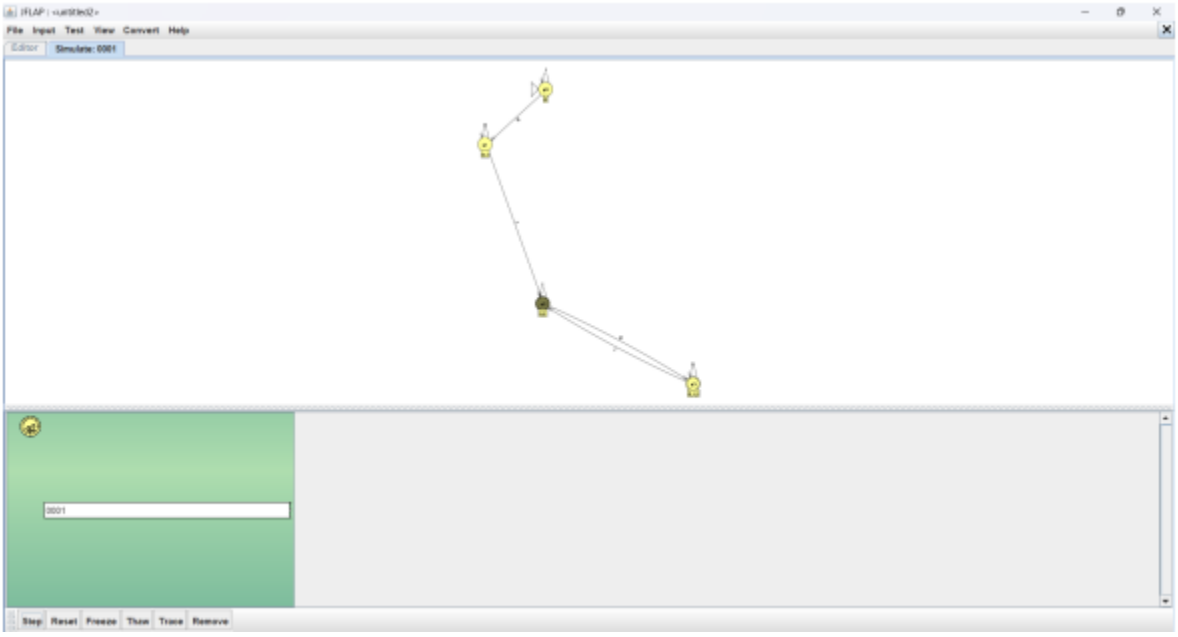


Equivalent DFA:

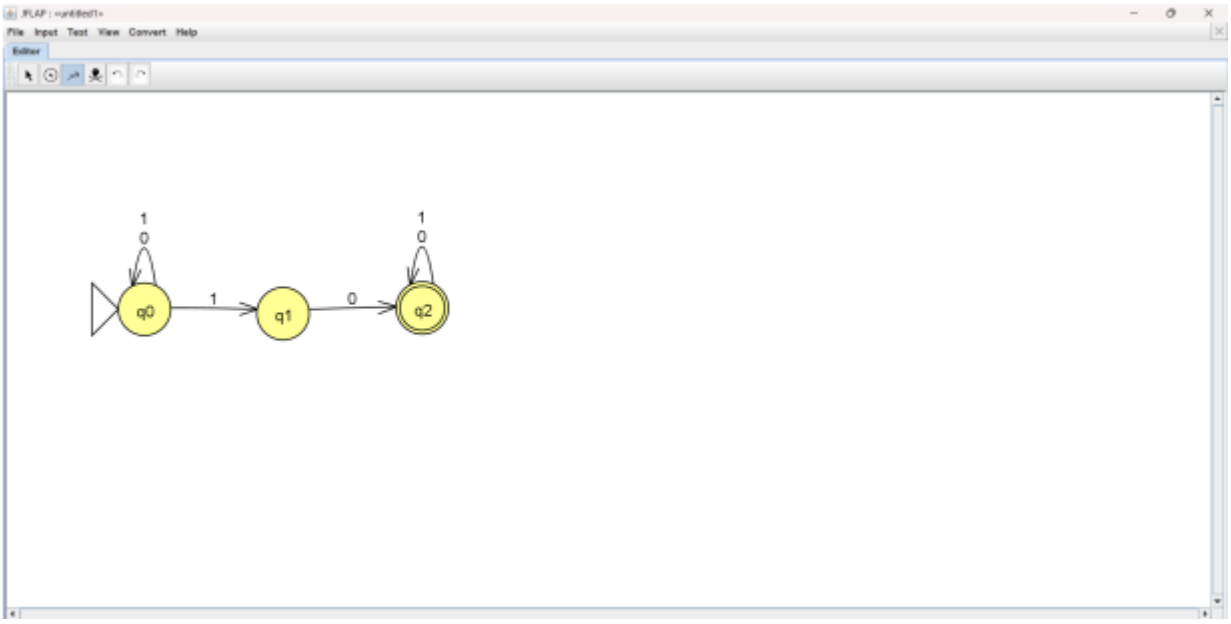


Test Case:

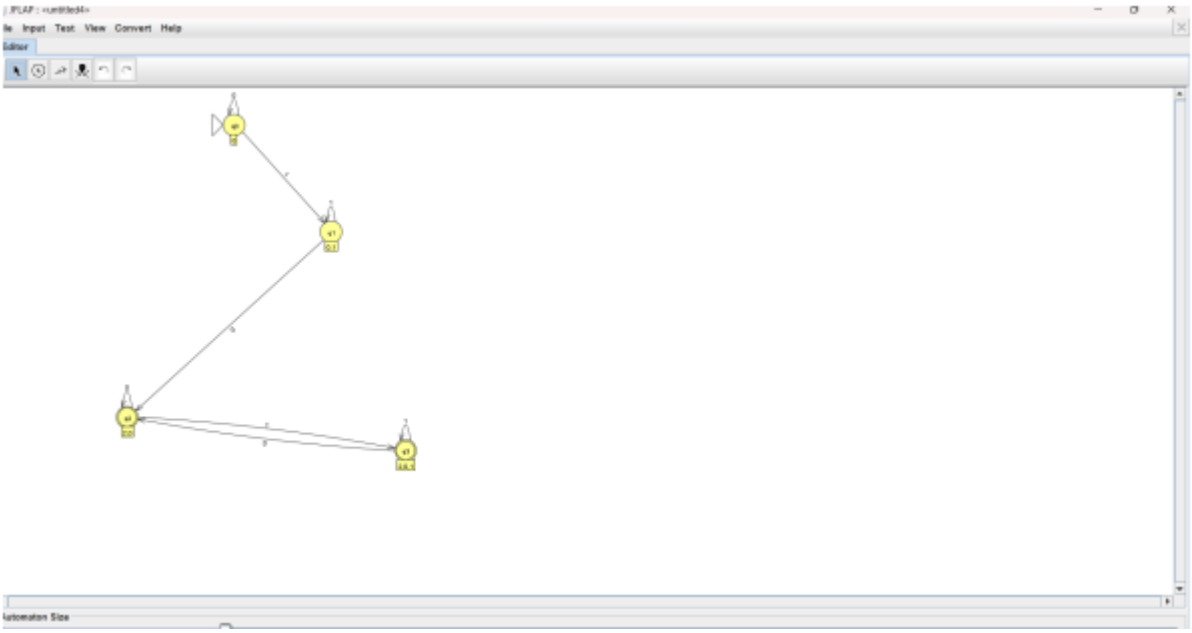
Input: 0001- accepted:



10. Construct the given NFA and convert it into an equivalent DFA using JFLAP containing 10 as a substring

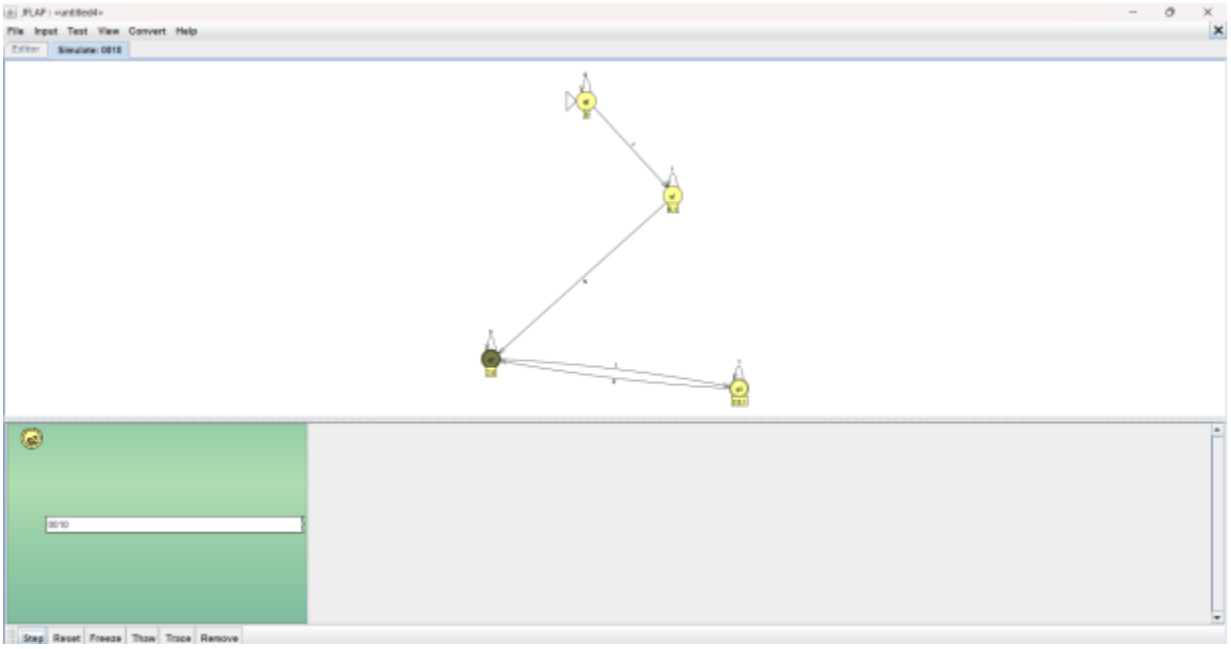


Equivalent DFA:



Test Cases:

Input: 0010 - accepted



Conclusion

These practicals demonstrate the construction and simulation of DFA and NFA, the recognition of binary strings based on specific patterns, and the conversion of NFA into an equivalent DFA using JFLAP. Testing different input strings helps verify the correctness of each automaton and develops an understanding of finite automata and regular languages.